Auditing and Building Claude Code Mods: 'Left Undone' Task Tracking

Researched 2026-10-12.

1. What the Reel Demonstrates

A video reel by creator Jason Rivard on Instagram (https://www.instagram.com/reel/DeJhQj8hkGy/) showcases a Claude Code modification named **Terminal Desk** (invoked via `/desk`).

- **Problem Addressed:** Claude often promises multi-part tasks in an initial plan (e.g., 1/2/3) but finishes its turn leaving items deferred or incomplete.
- **Detection Mechanism:** The creator claims the mod tracks incomplete tasks in real time across three sources: `claude-said` (detecting text like 'I did not run...' or 'for now'), `in-file` (unresolved `// TODO` or `# TODO` markers added during the session), and `dropped-model` (dropped planning milestones).
- **Interactive Dispatch:** A red 'Left undone' sidebar displays items with short letter IDs (`U1`, `U2`, etc.). Focusing the pane and pressing the letter fills Claude's prompt box with `You left this undone: <entry>. Do it now.` for rapid re-dispatch.
- **Distribution Funnel:** The mod is not hosted on a public repository; it is distributed exclusively via direct message when commenting 'DESK' on the post.

---

2. What Official Documentation Confirms

Official Anthropics documentation on the Claude Code Mods repository (https://github.com/anthropics/claude-code/tree/main/mods) validates several underlying architectural capabilities while clarifying runtime limits:

- **Plugin Architecture:** A Claude Code mod is packaged with a `.claude-plugin/plugin.json`, a `hooks/hooks.json` specifying the entry point, and TypeScript files under `hooks/` typed against engine declarations (`import type ... from 'claude-code'`).
- **Hook Execution:** Mods define a `register(on, options)` function that intercepts engine events using `($, e, next)`. Built-in reference mods include `sec-default`, `diff`, `telemetry`, and `agents-md`.
- **Terminal UI Capabilities:** The engine supports custom terminal panes and event-driven buttons via UI handlers like `ui.open` and `$.ui.press({ plugin, key })`, validating that sidebar panes like `/diff` and custom panels are natively feasible.
- **Testing & Local Running:** Official CLI tooling supports running mods from source via `claude --plugin-dir <path>` and running unit tests using `claude plugin test <path>` backed by `claude-code/testing` mocks (`mock.clock`, `mock.store`, `mock.env`).

---

3. Gaps, Unverified Claims & Security Risks

- **Unsandboxed Execution Risk:** Anthropic's mod architecture gives plugins direct access to the engine and host environment. Mods run with user privileges and can read environment variables, inspect

secrets, and execute shell commands. Installing arbitrary ZIP archives obtained via social media DMs poses critical security risks.

- **Unverified Statusline & Versions:** The reel's status bar displays 'Opus 5.5 ctx:32%', which is not an officially documented public model name. Similarly, automatic version updating via a 'global copy' from unverified paths is unsubstantiated.
- **API Stability:** Official docs explicitly label the hooks engine as **Early access**: 'hooks modules load only where function hooks are enabled, and the API these mods are written against may change between releases without notice.'
- **Heuristic Limitations:** Keyword matching on assistant outputs (`I did not run`, `for now`) is prone to false positives when Claude quotes user instructions, handles negative test assertions, or discusses hypothetical code.

---

4. Verified vs. Unverified Commands

Verified Commands (from Official Documentation (https://github.com/anthropics/claude-code/tree/main/mods))

- Run a mod from local source:

```
claude --plugin-dir mods/diff
```

- Run mod test suites:

```
claude plugin test mods/diff
```

- Typecheck mod hooks and declarations:

```
tsc -p mods/tsconfig.json
```

Unverified Reel Commands / Workflows

- Unpacking DM-sourced ZIP archives into `~/.claude/` or executing `/desk` (unverified and potentially dangerous).
- `claude plugin validate terminal-desk` (unverified command in official docs).

---

5. Reusable Reproduction Prompt

If you want to implement a clean-room, auditable version of a task-tracking mod using official mod interfaces, use this prompt with Claude Code:

```
Create a local Claude Code mod in directory 'undone-tracker' based on the official hooks
contract (register(on, options)) documented in anthropic's claude-code/mods.

Requirements:
1. Plugin structure:
   - undone-tracker/.claude-plugin/plugin.json
   - undone-tracker/hooks/hooks.json
```

```
   - undone-tracker/hooks/register.ts
   - undone-tracker/tests/register.test.ts
2. Hook behavior:
   - Register on command events to create a custom slash command '/undone'.
   - Intercept assistant output to parse unfulfilled tasks (e.g., detected TODO markers
written to files or explicit deferral phrases).
   - Provide a test suite using `claude-code/testing` asserting that the custom command
correctly registers and queries the session store.
3. Fail closed: Wrap all transcript parsers in try/catch blocks so errors never interrupt the
primary Claude Code session.
```

---

6. Common Mistakes

- **Executing Unvetted DM Code:** Never extract third-party plugins directly into active CLI configs without auditing the TypeScript source.
- **Missing Early Access Flags:** On certain environments, hooks will not mount unless function hooks are explicitly permitted by the local engine configuration.
- **Assuming Persistent Global State:** Mod memory that is not serialized through engine storage (`$.store`) will reset across sessions or after `/compact` operations.

Original sources

- https://www.instagram.com/reel/DeJhQj8hkGy/